

Práctica 2. Un servidor concurrente en C.

Miguel Ángel Dorado Maldonado 07/10/26.

STAGE 1.

Se implementa el loop principal del servidor manejando hijos, cerrando los descriptors adecuados.

Los 3 errores:

- accept no tenía todos los parámetros faltaba longitud de la dirección
- Familia de socket no soportada. Al ejecutar el programa se avisa de un fallo de compatibilidad.

Se trata de un socket de internet por lo tanto la familia a la que pertenece debe ser AF_INET que representa las conexiones por IPv4.

- No se controlan los posibles errores de escritura hacia el cliente. Si esto cierra la conexión los writes fallan.

Al conectar ambos clientes ambos reciben los mensajes en el tiempo esperado. No hay ningún fallo en el manejo de los hijos

Salida escrita en results.txt.

STAGE 2.

Se ejecuta el servidor y los clientes. Datos recogidos en results.txt.

El número de procesos es correcto. Cuenta servidor y clientes.

STAGE 3.

- --leave conecta al cliente y lo cierra. De esta forma podemos observar si los writes se controlan bien. Se muestra el mensaje de error esperado definido en la función serve-client.

Si no abandona el client no se muestra ningún mensaje de error.

- Procesos zombie. Los hijos no son controlados cuando mueren. Por esto quedan en estado zombie. (defunct)

Se ha implementado el control de hijos y se ha dejado comentado.

- Se mata al hijo. Al estar controlada la escritura al cliente se muestra el error si es que todavía no se ha escrito.

Realmente en el código sin modificar al matar a un hijo no bloquea a otros ya que la señal SIGPIPE que se lanza lo cierra tanto al proceso como a los descriptors por lo que no debería provocar ningún fallo.

Igualmente está bien controlar manualmente el error y llevar un log del sistema.